

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-214БВ-24

Студент: Семенченко Д. В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 27.11.24

Москва, 2025

Постановка задачи

Вариант 19.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в операционной системе macOS (или Linux).

В результате работы основной (родительский) процесс должен создать два дочерних процесса для решения задачи. Взаимодействие между процессами осуществляется исключительно через каналы (pipe). Необходимо корректно обрабатывать системные ошибки.

(19 вариант) Правило фильтрации: с вероятностью 80% строки отправляются в pipe1, иначе в pipe2. Дочерние процессы удаляют все гласные из строк.

Общий метод и алгоритм решения

Использованные системные вызовы:

- **pid_t fork(void);**

Создаёт дочерний процесс, являющийся копией родительского. После вызова оба процесса продолжают выполнение независимо, начиная со следующей инструкции. Используется дважды для создания двух дочерних процессов.

- **int pipe(int fd[2]);**

Создаёт однонаправленный канал межпроцессного взаимодействия. Возвращает два файловых дескриптора: fd[0] — для чтения, fd[1] — для записи. В программе создаются два канала: один для передачи строк первому дочернему процессу, второй — второму.

- **ssize_t read(int fd, void *buf, size_t count);**

Читает до count байт данных из файлового дескриптора fd в буфер buf. Используется для:

- чтения входных строк из STDIN_FILENO в родительском процессе,
- чтения данных из канала в дочерних процессах.

- **ssize_t write(int fd, const void *buf, size_t count);**

Записывает count байт из буфера buf в файловый дескриптор fd. Используется для:

- отправки строк в каналы из родительского процесса,
- записи результата (строк без гласных) в выходной файл из дочерних процессов.

- **int dup2(int oldfd, int newfd);**

Дублирует файловый дескриптор oldfd на newfd, предварительно закрыв newfd, если он был открыт. В дочерних процессах используется для перенаправления STDIN_FILENO на чтение из соответствующего канала.

- **pid_t waitpid(pid_t pid, int *status, int options);**

Приостанавливает выполнение родительского процесса до завершения указанных дочерних процессов. Гарантирует, что родитель не завершится раньше, чем дети обработают все данные.

- **int execv(const char *path, char *const argv[]);**

Заменяет текущий образ процесса новым, загружаемым из исполняемого файла по пути path, с передачей аргументов через argv. Используется в каждом дочернем процессе для запуска программы child.

- int close(int fd);

Закрывает файловый дескриптор fd, освобождая его для повторного использования. Применяется для закрытия ненужных концов каналов в каждом процессе (по принципу: «закрой, что не используешь»).

- int open(const char *pathname, int flags, mode_t mode);

Открывает или создаёт файл по пути pathname с заданными флагами (O_WRONLY | O_CREAT | O_TRUNC) и правами доступа (0644). Используется в дочерних процессах для создания/перезаписи выходных файлов.

Алгоритм работы программы:

Программа состоит из двух исполняемых компонентов: родительского процесса (parent) и дочернего процесса (child). Родитель управляет распределением данных, а дети выполняют фильтрацию и запись.

1. Инициализация и проверка аргументов

- Родительский процесс проверяет наличие двух аргументов командной строки: имён выходных файлов.
- При их отсутствии выводит сообщение об использовании и завершается с ошибкой.

2. Создание каналов связи

- Создаются два независимых канала с помощью pipe():
 - pipe1 — для передачи строк первому дочернему процессу,
 - pipe2 — для передачи строк второму дочернему процессу.

3. Создание дочерних процессов

- С помощью первого вызова `fork()` создаётся первый дочерний процесс.
- С помощью второго вызова `fork()` создаётся второй дочерний процесс.
- Всего после этого существует три процесса: один родительский и два дочерних.

4. Логика первого дочернего процесса

- Закрываются ненужные концы каналов: запись в `pipe1`, оба конца `pipe2`.
- С помощью `dup2()` стандартный ввод (`STDIN_FILENO`) перенаправляется на чтение из `pipe1[0]`.
- Выполняется `execv()` для запуска программы `./child` с передачей имени первого выходного файла в качестве аргумента.

5. Логика второго дочернего процесса

- Закрываются ненужные концы каналов: запись в `pipe2`, оба конца `pipe1`.
- С помощью `dup2()` стандартный ввод (`STDIN_FILENO`) перенаправляется на чтение из `pipe2[0]`.
- Выполняется `execv()` для запуска программы `./child` с передачей имени второго выходного файла.

6. Логика родительского процесса

- Закрываются концы каналов, предназначенные для чтения (`pipe1[0]`, `pipe2[0]`).
- В цикле читаются строки со стандартного ввода (`STDIN_FILENO`).
- Каждая строка с вероятностью 80% отправляется в `pipe1[1]`, иначе — в `pipe2[1]` (с использованием `rand()`).
- После окончания ввода (EOF) родитель закрывает оба канала записи и ожидает завершения обоих дочерних процессов с помощью `waitpid()`.

7. Обработка данных в дочернем процессе (child)

- Открывается указанный файл для записи с флагами `O_WRONLY | O_CREAT | O_TRUNC`.

- В цикле читаются данные из STDIN_FILENO (теперь связанного с каналом).
- Каждая строка обрабатывается: из неё удаляются все гласные буквы английского и русского алфавитов (в кодировке UTF-8).
- Результат записывается в выходной файл с помощью write().

8. Завершение работы

- Дочерние процессы завершаются после обработки всех данных и закрытия файлов.
- Родительский процесс завершается после ожидания завершения обоих потомков.

Код программы

parent.c

```
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

static void write_all(int fd, const char *buf, size_t len) {
    while (len > 0) {
        ssize_t w = write(fd, buf, len);
        if (w <= 0) _exit(1);
        buf += w;
        len -= w;
    }
}

int main(int argc, char *argv[]) {
    if (argc != 3) {
        write(STDERR_FILENO, "Usage: ./parent <file1> <file2>\n", 34);
        _exit(1);
    }

    int pipe1[2], pipe2[2];
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        write(STDERR_FILENO, "pipe failed\n", 12);
        _exit(1);
    }

    srand((unsigned int)time(NULL));

    pid_t pid1 = fork();
    if (pid1 == 0) {
        close(pipe1[1]); close(pipe2[0]); close(pipe2[1]);
```

```

dup2(pipe1[0], STDIN_FILENO);
close(pipe1[0]);
execl("./child", "./child", argv[1], (char *)NULL);
write(STDERR_FILENO, "exec child1 failed\n", 20);
_exit(1);
}

```

```

pid_t pid2 = fork();
if (pid2 == 0) {
    close(pipe2[1]); close(pipe1[0]); close(pipe1[1]);
    dup2(pipe2[0], STDIN_FILENO);
    close(pipe2[0]);
    execl("./child", "./child", argv[2], (char *)NULL);
    write(STDERR_FILENO, "exec child2 failed\n", 20);
    _exit(1);
}

```

```

close(pipe1[0]); close(pipe2[0]);

```

```

char buf[1024];
ssize_t n;
while ((n = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
    if (rand() % 100 < 80) {
        write_all(pipe1[1], buf, n);
    } else {
        write_all(pipe2[1], buf, n);
    }
}

```

```

close(pipe1[1]); close(pipe2[1]);
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

```

```

return 0;
}

```

child.c

```

#include <unistd.h>
#include <fcntl.h>
#include <string.h>

```

```

static const char* vowels[] = {
    "a", "e", "i", "o", "u",
    "A", "E", "I", "O", "U",
    "a", "e", "ё", "и", "o", "y", "ы", "э", "ю", "я",
    "A", "E", "Ё", "И", "O", "Y", "Ы", "Э", "Ю", "Я"
};
static const int num_vowels = sizeof(vowels) / sizeof(vowels[0]);

```

```

static int is_vowel_at(const char* buf, ssize_t len, ssize_t pos, ssize_t* out_len) {
    for (int i = 0; i < num_vowels; i++) {
        size_t vlen = strlen(vowels[i]);
        if (pos + (ssize_t)vlen <= len) {

```

```

        if (memcmp(&buf[pos], vowels[i], vlen) == 0) {
            *out_len = (ssize_t)vlen;
            return 1;
        }
    }
}
return 0;
}

int main(int argc, char *argv[]) {
    if (argc != 2) _exit(1);

    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) _exit(1);

    char buf[1024];
    char out[1024];
    ssize_t n;

    while ((n = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        ssize_t i = 0;
        size_t j = 0;
        while (i < n) {
            ssize_t vlen = 0;
            if (is_vowel_at(buf, n, i, &vlen)) {
                i += vlen;
            } else {
                out[j++] = buf[i];
                i++;
            }
        }
        if (j > 0) write(fd, out, j);
    }

    close(fd);
    return 0;
}

```

Протокол работы программы

Тестирование:

sedimav@MacBook-Pro-Dmitrij-2 LAB1 % make

clang -Wall -Wextra -O2 -o parent parent.c

clang -Wall -Wextra -O2 -o child child.c

sedimav@MacBook-Pro-Dmitrij-2 LAB1 % cat > input.txt << EOF

heredoc> >....

Cherry

Добрый

Вечер

Привет

Мир

Test

Тест

Линия

Юла

Ёжик

University

Education

Образование

Programming

Code

Код

Linux

macOS

Russian

Русский

English

Английский

EOF

```
sedimav@MacBook-Pro-Dmitrij-2 LAB1 % cat input.txt | ./parent out1.txt out2.txt
```

```
sedimav@MacBook-Pro-Dmitrij-2 LAB1 % cat out1.txt
```

Hll

Wrld

ppl

блк

Впп

Бнн

Chrry

Дбрй

Вчр

Првт

Мр

Tst

Тст

Лн

л

жк

nvrsty

dctn

брзвн

Prgrmmng

Cd

Кд

Lnx

mcS

Rssn

Рсскй

nglsh

нглйскй

sedimav@MacBook-Pro-Dmitrij-2 LAB1 % cat out2.txt

sedimav@MacBook-Pro-Dmitrij-2 LAB1 % cat input.txt | sudo dtruss -f ./parent trace1.txt trace2.txt 2> dtrace.log
Password:

sedimav@MacBook-Pro-Dmitrij-2 LAB1 % cat trace1.txt

sedimav@MacBook-Pro-Dmitrij-2 LAB1 % cat trace2.txt

Hll

Wrld

ррl

блк

Вnn

Бnn

Chrry

Дбрй

Вчр

Првт

Мр

Tst

Тст

Лн

л

жк

nvrsty

dctn

брзвн

Prgrmmng

Cd

Кд

Lnx

mcS

Rssn

Рсскй

nglsh

нглйскй

dtrace:

dtrace: system integrity protection is on, some features will not be available

PID/THRD SYSCALL(args) = return

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

3470/0x68a03: fork() = 0 0

3470/0x68a03: csrctl(0x0, 0x7FF7B207137C, 0x4) = -1 1

3470/0x68a03: proc_info(0xF, 0xD8E, 0x0) = 0 0

3470/0x68a03: thread_selfid(0x0, 0x0, 0x0) = 428547 0

3470/0x68a03: fsgetpath(0x7FF7B20713D0, 0x400, 0x7FF7B2071820) = 14 0

3470/0x68a03: shared_region_check_np(0x7FF7B20719C8, 0x0, 0x0) = 0 0

3470/0x68a03: getpid(0x0, 0x0, 0x0) = 3470 0

3470/0x68a03: munmap(0x11C9F4000, 0x9E000) = 0 0

3470/0x68a03: munmap(0x11CA92000, 0x7000) = 0 0

3470/0x68a03: munmap(0x11CA99000, 0x22000) = 0 0

```

3470/0x68a03: munmap(0x11CABB000, 0x4000)          = 0 0
3470/0x68a03: munmap(0x11CABF000, 0x0)             = -1 22
3470/0x68a03: munmap(0x11CABF000, 0x54000)          = 0 0
3470/0x68a03: open(".\0", 0x100000, 0x0)           = 3 0
3470/0x68a03: fcntl(0x3, 0x32, 0x7FF7B2070B10)      = 0 0
3470/0x68a03: close(0x3)                           = 0 0
3470/0x68a03: fsgetpath(0x7FF7B2070B20, 0x400, 0x7FF7B2070B08) = 40 0
3470/0x68a03: fsgetpath(0x7FF7B2070B20, 0x400, 0x7FF7B2070B08) = 14 0
3470/0x68a03: csrctl(0x0, 0x7FF7B2070F2C, 0x4)      = -1 1
3470/0x68a03: __mac_syscall(0x7FF8117C20D6, 0x2, 0x7FF7B2070D70) = 0 0
3470/0x68a03: csrctl(0x0, 0x7FF7B2070F2C, 0x4)      = -1 1
3470/0x68a03: __mac_syscall(0x7FF8117BEF76, 0x5A, 0x7FF7B2070EC0) = 0 0
3470/0x68a03: open("\0", 0x20100000, 0x0)          = 3 0
3470/0x68a03: openat(0x3, "System/Cryptexes/OS\0", 0x100000, 0x0) = 4 0
3470/0x68a03: dup(0x4, 0x0, 0x0)                   = 5 0
3470/0x68a03: fstatat64(0x4, 0x7FF7B206FC51, 0x7FF7B2070050) = 0 0
3470/0x68a03: openat(0x4, "System/Library/dyld/\0", 0x100000, 0x0) = 6 0
3470/0x68a03: fcntl(0x6, 0x32, 0x7FF7B206FCE0)      = 0 0
3470/0x68a03: dup(0x6, 0x0, 0x0)                   = 7 0
3470/0x68a03: dup(0x5, 0x0, 0x0)                   = 8 0
3470/0x68a03: close(0x3)                           = 0 0
3470/0x68a03: close(0x5)                           = 0 0
3470/0x68a03: close(0x4)                           = 0 0
3470/0x68a03: close(0x6)                           = 0 0
3470/0x68a03: __mac_syscall(0x7FF8117C20D6, 0x2, 0x7FF7B20708B0) = 0 0
3470/0x68a03: shared_region_check_np(0x7FF7B20709B8, 0x0, 0x0) = 0 0

```

3470/0x68a03: fsgetpath(0x7FF7B2070B60, 0x400, 0x7FF7B2070AEC) = 83 0

3470/0x68a03: fcntl(0x8, 0x32, 0x7FF7B2070B60) = 0 0

3470/0x68a03: close(0x8) = 0 0

3470/0x68a03: close(0x7) = 0 0

3470/0x68a03: stat64("/Users/sedimav/Desktop/^320\236\320\241/LAB1/parent\0", 0x7FF7B20706D0, 0x0) = 0 0

3470/0x68a03: open("/Users/sedimav/Desktop/^320\236\320\241/LAB1/parent\0", 0x0, 0x0) = 3 0

3470/0x68a03: mmap(0x0, 0x2340, 0x1, 0x40002, 0x3, 0x0) = 0x10DE91000 0

3470/0x68a03: fcntl(0x3, 0x32, 0x7FF7B20707E0) = 0 0

3470/0x68a03: munmap(0x10DE91000, 0x2340) = 0 0

3470/0x68a03: close(0x3) = 0 0

3470/0x68a03: stat64("/Users/sedimav/Desktop/^320\236\320\241/LAB1/parent\0", 0x7FF7B2070C40, 0x0) = 0 0

3470/0x68a03: stat64("/usr/lib/libSystem.B.dylib\0", 0x7FF7B206FBF0, 0x0) = -1 2

3470/0x68a03: stat64("/System/Volumes/Preboot/Cryptexes/OS/usr/lib/libSystem.B.dylib\0", 0x7FF7B206FBA0, 0x0) = -1 2

3470/0x68a03: open("/Users/sedimav/Desktop/^320\236\320\241/LAB1/parent\0", 0x0, 0x0) = 3 0

3470/0x68a03: __mac_syscall(0x7FF8117C20D6, 0x2, 0x7FF7B206DA50) = 0 0

3470/0x68a03: map_with_linking_np(0x7FF7B206D9F0, 0x1, 0x7FF7B206DA20) = -1 22

3470/0x68a03: close(0x3) = 0 0

3470/0x68a03: mprotect(0x10DE8F000, 0x1000, 0x1) = 0 0

3470/0x68a03: getfsstat64(0x0, 0x0, 0x2) = 11 0

3470/0x68a03: getfsstat64(0x10DEB1060, 0x5D28, 0x2) = 11 0

3470/0x68a03: getattrlist("/^0", 0x7FF7B206D2B0, 0x7FF7B206D220) = 0 0

3470/0x68a03: open("/dev/dtracehelper\0", 0x2, 0x0) = 3 0

3470/0x68a03: ioctl(0x3, 0x80086804, 0x7FF7B206D198) = 0 0

3470/0x68a03: close(0x3) = 0 0

3470/0x68a03: shared_region_check_np(0xFFFFFFFFFFFFFFFF, 0x0, 0x0) = 0 0

3470/0x68a03: access("/AppleInternal/XBS/.isChrooted\0", 0x0, 0x0) = -1 2

3470/0x68a03: bsdthread_register(0x7FF811B03848, 0x7FF811B03834, 0x2000) =
1073742303 0

3470/0x68a03: getpid(0x0, 0x0, 0x0) = 3470 0

3470/0x68a03: shm_open(0x7FF811996CE1, 0x0, 0x11995121) = 3 0

3470/0x68a03: fstat64(0x3, 0x7FF7B206D930, 0x0) = 0 0

3470/0x68a03: mmapdtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel
access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at
DIF offset 28

dtrace: error on enabled probe ID 1719 (ID 177: syscall::read:return): invalid kernel access in action #13 at
DIF offset 68

dtrace: error on enabled probe ID 1717 (ID 179: syscall::write:return): invalid kernel access in action #13 at
DIF offset 68

dtrace: error on enabled probe ID 1719 (ID 177: syscall::read:return): invalid kernel access in action #13 at
DIF offset 68

dtrace: error on enabled probe ID 1694 (ID 289: syscall::execve:return): invalid address (0x10de8e78c) in
action #12 at DIF offset 12

dtrace: error on enabled probe ID 1694 (ID 289: syscall::execve:return): invalid address (0x10de8e78c) in
action #12 at DIF offset 12

(0x0, 0x7000, 0x1, 0x40001, 0x3, 0x0) = 0x10DE93000 0

3470/0x68a03: close(0x3) = 0 0

3470/0x68a03: csops(0xD8E, 0x0, 0x7FF7B206DA64) = 0 0

3470/0x68a03: ioctl(0x2, 0x4004667A, 0x7FF7B206D9F4) = -1 25

3470/0x68a03: ioctl(0x2, 0x40487413, 0x7FF7B206D9F8) = -1 25

3470/0x68a03: mprotect(0x10DE9F000, 0x1000, 0x0) = 0 0

3470/0x68a03: mprotect(0x10DEA6000, 0x1000, 0x0) = 0 0

3470/0x68a03: mprotect(0x10DEA7000, 0x1000, 0x0) = 0 0

3470/0x68a03: mprotect(0x10DEAE000, 0x1000, 0x0)	= 0 0
3470/0x68a03: mprotect(0x10DE9A000, 0xC8, 0x1)	= 0 0
3470/0x68a03: mprotect(0x10DE9A000, 0xC8, 0x3)	= 0 0
3470/0x68a03: mprotect(0x10DE9A000, 0xC8, 0x1)	= 0 0
3470/0x68a03: mprotect(0x10DEAF000, 0x1000, 0x1)	= 0 0
3470/0x68a03: mprotect(0x10DEB0000, 0xC8, 0x1)	= 0 0
3470/0x68a03: mprotect(0x10DEB0000, 0xC8, 0x3)	= 0 0
3470/0x68a03: mprotect(0x10DEB0000, 0xC8, 0x1)	= 0 0
3470/0x68a03: mprotect(0x10DE9A000, 0xC8, 0x3)	= 0 0
3470/0x68a03: mprotect(0x10DE9A000, 0xC8, 0x1)	= 0 0
3470/0x68a03: mprotect(0x10DEAF000, 0x1000, 0x3)	= 0 0
3470/0x68a03: mprotect(0x10DEAF000, 0x1000, 0x1)	= 0 0
3470/0x68a03: issetugid(0x0, 0x0, 0x0)	= 0 0
3470/0x68a03: getentropy(0x7FF7B206D0C0, 0x20, 0x0)	= 0 0
3470/0x68a03: getattrlist("/Users/sedimav/Desktop/\320\236\320\241/LAB1/parent\0", 0x7FF7B206D8F0, 0x7FF7B206D90C)	= 0 0
3470/0x68a03: access("/Users/sedimav/Desktop/\320\236\320\241/LAB1\0", 0x4, 0x0)	= 0 0
3470/0x68a03: open("/Users/sedimav/Desktop/\320\236\320\241/LAB1\0", 0x0, 0x0)	= 3 0
3470/0x68a03: fstat64(0x3, 0x7F7DDB704440, 0x0)	= 0 0
3470/0x68a03: csrctl(0x0, 0x7FF7B206DAFC, 0x4)	= -1 1
3470/0x68a03: fgetattrlist(0x3, 0x7FF7B206DBB0, 0x7FF7B206DB10)	= 0 0
3470/0x68a03: __mac_syscall(0x7FF81F0EFD45, 0x2, 0x7FF7B206DB10)	= 0 0
3470/0x68a03: fcntl(0x3, 0x32, 0x7FF7B206D800)	= 0 0
3470/0x68a03: close(0x3)	= 0 0
3470/0x68a03: open("/Users/sedimav/Desktop/\320\236\320\241/LAB1/Info.plist\0", 0x0, 0x0)	= -
1 2	
3470/0x68a03: proc_info(0x2, 0xD8E, 0xD)	= 64 0

3470/0x68a03: csops_audittoken(0xD8E, 0x10, 0x7FF7B206DB20)	= -1 22
3470/0x68a03: pipe(0x0, 0x0, 0x0)	= 3 0
3470/0x68a03: pipe(0x0, 0x0, 0x0)	= 5 0
3470/0x68a03: fork()	= 3471 0
3471/0x68a0a: fork()	= 0 0
3471/0x68a0a: thread_selfid(0x0, 0x0, 0x0)	= 428554 0
3471/0x68a0a: bsdthread_register(0x7FF811B03848, 0x7FF811B03834, 0x2000)	= -1 22
3471/0x68a0a: mprotect(0x10DEB0000, 0xC8, 0x3)	= 0 0
3471/0x68a0a: mprotect(0x10DEB0000, 0xC8, 0x1)	= 0 0
3471/0x68a0a: close(0x4)	= 0 0
3471/0x68a0a: close(0x5)	= 0 0
3471/0x68a0a: close(0x6)	= 0 0
3470/0x68a03: fork()	= 3472 0
3471/0x68a0a: dup2(0x3, 0x0, 0x0)	= 0 0
3471/0x68a0a: close(0x3)	= 0 0
3472/0x68a0b: fork()	= 0 0
3472/0x68a0b: thread_selfid(0x0, 0x0, 0x0)	= 428555 0
3470/0x68a03: close(0x3)	= 0 0
3472/0x68a0b: bsdthread_register(0x7FF811B03848, 0x7FF811B03834, 0x2000)	= -1 22
3470/0x68a03: close(0x5)	= 0 0
3472/0x68a0b: mprotect(0x10DEB0000, 0xC8, 0x3)	= 0 0
3472/0x68a0b: mprotect(0x10DEB0000, 0xC8, 0x1)	= 0 0
3470/0x68a03: close(0x4)	= 0 0
3470/0x68a03: close(0x6)	= 0 0
3472/0x68a0b: close(0x6)	= 0 0
3472/0x68a0b: close(0x3)	= 0 0

3472/0x68a0b: close(0x4) = 0 0

3472/0x68a0b: dup2(0x5, 0x0, 0x0) = 0 0

3472/0x68a0b: close(0x5) = 0 0

3471/0x68a0c: fork() = 0 0

3471/0x68a0c: mprotect(0x114C8A000, 0x7000, 0x1) = 0 0

3471/0x68a0c: thread_selfid(0x0, 0x0, 0x0) = 428556 0

3471/0x68a0c: fsgetpath(0x7FF7B9BDA3F0, 0x400, 0x7FF7B9BDA840) = 14 0

3471/0x68a0c: shared_region_check_np(0x7FF7B9BDA9E8, 0x0, 0x0) = 0 0

3471/0x68a0c: getpid(0x0, 0x0, 0x0) = 3471 0

3471/0x68a0c: csrctl(0x0, 0x7FF7B9BDA39C, 0x4) = -1 1

3471/0x68a0c: proc_info(0xF, 0xD8F, 0x0) = 0 0

3471/0x68a0c: thread_selfid(0x0, 0x0, 0x0) = 428556 0

3471/0x68a0c: fsgetpath(0x7FF7B9BDA3F0, 0x400, 0x7FF7B9BDA840) = 14 0

3471/0x68a0c: shared_region_check_np(0x7FF7B9BDA9E8, 0x0, 0x0) = 0 0

3472/0x68a0d: fork() = 0 0

3471/0x68a0c: getpidtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::syscall:return): invalid kernel access in action #11 at DIF offset 28

d(0x0, 0x0, 0x0) = 3471 0

3472/0x68a0d: mprotect(0x109751000, 0x7000, 0x1) = 0 0

3472/0x68a0d: thread_selfid(0x0, 0x0, 0x0) = 428557 0

3472/0x68a0d: fsgetpath(0x7FF7B7B063F0, 0x400, 0x7FF7B7B06840) = 14 0

3471/0x68a0c: munmap(0x114BEC000, 0x9E000) = 0 0

3472/0x68a0d: shared_region_check_np(0x7FF7B7B069E8, 0x0, 0x0) = 0 0

3471/0x68a0c: munmap(0x114C8A000, 0x7000) = 0 0

3471/0x68a0c: munmap(0x114C91000, 0x22000) = 0 0

3471/0x68a0c: munmap(0x114CB3000, 0x4000) = 0 0

3471/0x68a0c: munmap(0x114CB7000, 0x0) = -1 22

3471/0x68a0c: munmap(0x114CB7000, 0x54000) = 0 0

3472/0x68a0d: getpid(0x0, 0x0, 0x0) = 3472 0

3471/0x68a0c: open("./0", 0x100000, 0x0) = 3 0

3471/0x68a0c: fcntl(0x3, 0x32, 0x7FF7B9BD9B30) = 0 0

3471/0x68a0c: close(0x3) = 0 0

3471/0x68a0c: fsgetpath(0x7FF7B9BD9B40, 0x400, 0x7FF7B9BD9B28) = 39 0

3472/0x68a0d: csrctl(0x0, 0x7FF7B7B0639C, 0x4) = -1 1

3471/0x68a0c: fsgetpath(0x7FF7B9BD9B40, 0x400, 0x7FF7B9BD9B28) = 14 0

3471/0x68a0c: csrctl(0x0, 0x7FF7B9BD9F4C, 0x4) = -1 1

3471/0x68a0c: __mac_syscall(0x7FF8117C20D6, 0x2, 0x7FF7B9BD9D90) = 0 0

3471/0x68a0c: csrctl(0x0, 0x7FF7B9BD9F4C, 0x4) = -1 1

3471/0x68a0c: __mac_syscall(0x7FF8117BEF76, 0x5A, 0x7FF7B9BD9EE0) = 0 0

3472/0x68a0d: proc_info(0xF, 0xD90, 0x0) = 0 0

3471/0x68a0c: open("/0", 0x20100000, 0x0) = 3 0

3472/0x68a0d: thread_selfid(0x0, 0x0, 0x0)	= 428557 0
3471/0x68a0c: openat(0x3, "System/Cryptexes/OS\0", 0x100000, 0x0)	= 4 0
3471/0x68a0c: dup(0x4, 0x0, 0x0)	= 5 0
3472/0x68a0d: fsgetpath(0x7FF7B7B063F0, 0x400, 0x7FF7B7B06840)	= 14 0
3471/0x68a0c: fstatat64(0x4, 0x7FF7B9BD8C71, 0x7FF7B9BD9070)	= 0 0
3472/0x68a0d: shared_region_check_np(0x7FF7B7B069E8, 0x0, 0x0)	= 0 0
3471/0x68a0c: openat(0x4, "System/Library/dyld/\0", 0x100000, 0x0)	= 6 0
3471/0x68a0c: fcntl(0x6, 0x32, 0x7FF7B9BD8D00)	= 0 0
3471/0x68a0c: dup(0x6, 0x0, 0x0)	= 7 0
3472/0x68a0d: getpid(0x0, 0x0, 0x0)	= 3472 0
3471/0x68a0c: dup(0x5, 0x0, 0x0)	= 8 0
3471/0x68a0c: close(0x3)	= 0 0
3471/0x68a0c: close(0x5)	= 0 0
3471/0x68a0c: close(0x4)	= 0 0
3471/0x68a0c: close(0x6)	= 0 0
3471/0x68a0c: __mac_syscall(0x7FF8117C20D6, 0x2, 0x7FF7B9BD98D0)	= 0 0
3471/0x68a0c: shared_region_check_np(0x7FF7B9BD99D8, 0x0, 0x0)	= 0 0
3471/0x68a0c: fsgetpath(0x7FF7B9BD9B80, 0x400, 0x7FF7B9BD9B0C)	= 83 0
3472/0x68a0d: munmap(0x1096B3000, 0x9E000)	= 0 0
3471/0x68a0c: fcntl(0x8, 0x32, 0x7FF7B9BD9B80)	= 0 0
3471/0x68a0c: close(0x8)	= 0 0
3472/0x68a0d: munmap(0x109751000, 0x7000)	= 0 0
3471/0x68a0c: close(0x7)	= 0 0
3472/0x68a0d: munmap(0x109758000, 0x22000)	= 0 0
3472/0x68a0d: munmap(0x10977A000, 0x4000)	= 0 0
3472/0x68a0d: munmap(0x10977E000, 0x0)	= -1 22

```

3472/0x68a0d: munmap(0x10977E000, 0x54000)          = 0 0

3471/0x68a0c: stat64("/Users/sedimav/Desktop/\320\236\320\241/LAB1/child\0", 0x7FF7B9BD96F0,
0x0)          = 0 0

3471/0x68a0c: open("/Users/sedimav/Desktop/\320\236\320\241/LAB1/child\0", 0x0, 0x0)          = 3
0

3472/0x68a0d: open(".\0", 0x100000, 0x0)            = 3 0

3471/0x68a0c: mmap(0x0, 0x2290, 0x1, 0x40002, 0x3, 0x0)          = 0x106328000 0

3472/0x68a0d: fcntl(0x3, 0x32, 0x7FF7B7B05B30)        = 0 0

3472/0x68a0d: close(0x3)          = 0 0

3471/0x68a0c: fcntl(0x3, 0x32, 0x7FF7B9BD9800)          = 0 0

3472/0x68a0d: fsgetpath(0x7FF7B7B05B40, 0x400, 0x7FF7B7B05B28)      = 39 0

3471/0x68a0c: munmap(0x106328000, 0x2290)          = 0 0

3471/0x68a0c: close(0x3)          = 0 0

3472/0x68a0d: fsgetpath(0x7FF7B7B05B40, 0x400, 0x7FF7B7B05B28)      = 14 0

3471/0x68a0c: stat64("/Users/sedimav/Desktop/\320\236\320\241/LAB1/child\0", 0x7FF7B9BD9C60,
0x0)          = 0 0

3472/0x68a0d: csrctl(0x0, 0x7FF7B7B05F4C, 0x4)          = -1 1

3472/0x68a0d: __mac_syscall(0x7FF8117C20D6, 0x2, 0x7FF7B7B05D90)      = 0 0

3472/0x68a0d: csrctl(0x0, 0x7FF7B7B05F4C, 0x4)          = -1 1

3471/0x68a0c: stat64("/usr/lib/libSystem.B.dylib\0", 0x7FF7B9BD8C10, 0x0)      = -1 2

3471/0x68a0c: stat64("/System/Volumes/Preboot/Cryptexes/OS/usr/lib/libSystem.B.dylib\0",
0x7FF7B9BD8BC0, 0x0)          = -1 2

3472/0x68a0d: __mac_syscall(0x7FFdtrace: error on enabled probe ID 1747 (ID 575:
syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at
DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at
DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at
DIF offset 28

```

```

8117BEF76, 0x5A, 0x7FF7B7B05EE0)          = 0 0

3472/0x68a0d: open("\0", 0x20100000, 0x0)      = 3 0

3472/0x68a0d: openat(0x3, "System/Cryptexes/OS\0", 0x100000, 0x0)      = 4 0

3472/0x68a0d: dup(0x4, 0x0, 0x0)              = 5 0

3472/0x68a0d: fstatat64(0x4, 0x7FF7B7B04C71, 0x7FF7B7B05070)          = 0 0

3472/0x68a0d: openat(0x4, "System/Library/dyld\0", 0x100000, 0x0)      = 6 0

3472/0x68a0d: fcntl(0x6, 0x32, 0x7FF7B7B04D00)          = 0 0

3472/0x68a0d: dup(0x6, 0x0, 0x0)              = 7 0

3472/0x68a0d: dup(0x5, 0x0, 0x0)              = 8 0

3472/0x68a0d: close(0x3)                      = 0 0

3472/0x68a0d: close(0x5)                      = 0 0

3472/0x68a0d: close(0x4)                      = 0 0

3472/0x68a0d: close(0x6)                      = 0 0

3472/0x68a0d: __mac_syscall(0x7FF8117C20D6, 0x2, 0x7FF7B7B058D0)      = 0 0

3472/0x68a0d: shared_region_check_np(0x7FF7B7B059D8, 0x0, 0x0)          = 0 0

3472/0x68a0d: fsgetpath(0x7FF7B7B05B80, 0x400, 0x7FF7B7B05B0C)          = 83 0

3472/0x68a0d: fcntl(0x8, 0x32, 0x7FF7B7B05B80)          = 0 0

3472/0x68a0d: close(0x8)                      = 0 0

3472/0x68a0d: close(0x7)                      = 0 0

3472/0x68a0d: stat64("/Users/sedimav/Desktop/\320\236\320\241/LAB1/child\0", 0x7FF7B7B056F0,
0x0)          = 0 0

3472/0x68a0d: open("/Users/sedimav/Desktop/\320\236\320\241/LAB1/child\0", 0x0, 0x0)      = 3
0

3472/0x68a0d: mmap(0x0, 0x2290, 0x1, 0x40002, 0x3, 0x0)          = 0x1083FC000 0

3472/0x68a0d: fcntl(0x3, 0x32, 0x7FF7B7B05800)          = 0 0

3472/0x68a0d: munmap(0x1083FC000, 0x2290)          = 0 0

3472/0x68a0d: close(0x3)                      = 0 0

```

```

3472/0x68a0d: stat64("/Users/sedimav/Desktop/320\236\320\241/LAB1/child\0", 0xFF7B7B05C60,
0x0)          = 0 0

3471/0x68a0c: open("/Users/sedimav/Desktop/320\236\320\241/LAB1/child\0", 0x0, 0x0)          = 3
0

3472/0x68a0d: stat64("/usr/lib/libSystem.B.dylib\0", 0xFF7B7B04C10, 0x0)          = -1 2

3471/0x68a0c: __mac_syscall(0xFF8117C20D6, 0x2, 0xFF7B9BD6A70)          = 0 0

3472/0x68a0d: stat64("/System/Volumes/Preboot/Cryptexes/OS/usr/lib/libSystem.B.dylib\0",
0xFF7B7B04BC0, 0x0)          = -1 2

3471/0x68a0c: map_with_linking_np(0xFF7B9BD6A40, 0x1, 0xFF7B9BD6A70)          = -1 22

3471/0x68a0c: close(0x3)          = 0 0

3471/0x68a0c: mprotect(0x106326000, 0x1000, 0x1)          = 0 0

3471/0x68a0c: getfsstat64(0x0, 0x0, 0x2)          = 11 0

3471/0x68a0c: getfsstat64(0x106348060, 0x5D28, 0x2)          = 11 0

3471/0x68a0c: getattrlist("/\0", 0xFF7B9BD62D0, 0xFF7B9BD6240)          = 0 0

3471/0x68a0c: open("/dev/dtracehelper\0", 0x2, 0x0)          = 3 0

3472/0x68a0d: open("/Users/sedimav/Desktop/320\236\320\241/LAB1/child\0", 0x0, 0x0)          = 3
0

3472/0x68a0d: __mac_syscall(0xFF8117C20D6, 0x2, 0xFF7B7B02A70)          = 0 0

3472/0x68a0d: map_with_linking_np(0xFF7B7B02A40, 0x1, 0xFF7B7B02A70)          = -1 22

3472/0x68a0d: close(0x3)          = 0 0

3472/0x68a0d: mprotect(0x1083FA000, 0x1000, 0x1)          = 0 0

3472/0x68a0d: getfsstat64(0x0, 0x0, 0x2)          = 11 0

3472/0x68a0d: getfsstat64(0x10841C060, 0x5D28, 0x2)          = 11 0

3472/0x68a0d: getattrlist("/\0", 0xFF7B7B022D0, 0xFF7B7B02240)          = 0 0

3471/0x68a0c: ioctl(0x3, 0x80086804, 0xFF7B9BD61B8)          = 0 0

3471/0x68a0c: close(0x3)          = 0 0

3471/0x68a0c: shared_region_check_np(0xFFFFFFFFFFFFFFFF, 0x0, 0x0)          = 0 0

3472/0x68a0d: open("/dev/dtracehelper\0", 0x2, 0x0)          = 3 0

```

```

3471/0x68a0c: access("/AppleInternal/XBS/.isChrooted\0", 0x0, 0x0)          = -1 2

3471/0x68a0c: bsdthread_register(0x7FF811B03848, 0x7FF811B03834, 0x2000)    =
1073742303 0

3471/0x68a0c: getpid(0x0, 0x0, 0x0)          = 3471 0

3471/0x68a0c: shm_open(0x7FF811996CE1, 0x0, 0x11995121)          = 3 0

3471/0x68a0c: fstat64(0x3, 0x7FF7B9BD6950, 0x0)          = 0 0

3471/0x68a0c: mmap(0x0, 0x7000, 0x1, 0x40001, 0x3, 0x0)          = 0x10632A000 0

3471/0x68a0c: close(0x3)          = 0 0

3471/0x68a0c: csops(0xD8F, 0x0, 0x7FF7B9BD6A84)          = 0 0

3471/0x68a0c: ioctl(0x2, 0x4004667A, 0x7FF7B9BD6A14)          = -1 25

3471/0x68a0c: ioctl(0x2, 0x40487413, 0x7FF7B9BD6A18)          = -1 25

3472/0x68a0d: ioctl(0x3, 0x80086804, 0x7FF7B7B021B8)          = 0 0

3472/0x68a0d: close(0x3)          = 0 0

3471/0x68a0c: mprotect(0x106336000, 0x1000, 0x0)          = 0 0

3472/0x68a0d: shared_region_check_np(0xFFFFFFFFFFFFFFFF, 0x0, 0x0)          = 0 0

3471/0x68a0c: mprotect(0x10633D000, 0x1000, 0x0)          = 0 0

3471/0x68a0c: mprotect(0x10633E000, 0x1000, 0x0)          = 0 0
dtrace: error on enabled probe ID
1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at
DIF offset 28

3471/0x68a0c: mprotect(0x106345000, 0x1000, 0x0)          = 0 0

3472/0x68a0d: access("/AppleInternal/XBS/.isChrooted\0", 0x0, 0x0)          = -1 2

3471/0x68a0c: mprotect(0x106331000, 0xC8, 0x1)          = 0 0

3471/0x68a0c: mprotect(0x106331000, 0xC8, 0x3)          = 0 0

3471/0x68a0c: mprotect(0x106331000, 0xC8, 0x1)          = 0 0

3472/0x68a0d: bsdthread_register(0x7FF811B03848, 0x7FF811B03834, 0x2000)    =
1073742303 0

```


3471/0x68a0c: mprotect(0x106346000, 0x1000, 0x1)	= 0 0
3471/0x68a0c: mprotect(0x106347000, 0xC8, 0x1)	= 0 0
3471/0x68a0c: mprotect(0x106347000, 0xC8, 0x3)	= 0 0
3471/0x68a0c: mprotect(0x106347000, 0xC8, 0x1)	= 0 0
3472/0x68a0d: getpid(0x0, 0x0, 0x0)	= 3472 0
3471/0x68a0c: mprotect(0x106331000, 0xC8, 0x3)	= 0 0
3472/0x68a0d: shm_open(0x7FF811996CE1, 0x0, 0x11995121)	= 3 0
3471/0x68a0c: mprotect(0x106331000, 0xC8, 0x1)	= 0 0
3472/0x68a0d: fstat64(0x3, 0x7FF7B7B02950, 0x0)	= 0 0
3471/0x68a0c: mprotect(0x106346000, 0x1000, 0x3)	= 0 0
3471/0x68a0c: mprotect(0x106346000, 0x1000, 0x1)	= 0 0
3472/0x68a0d: mmap(0x0, 0x7000, 0x1, 0x40001, 0x3, 0x0)	= 0x1083FE000 0
3472/0x68a0d: close(0x3)	= 0 0
3472/0x68a0d: csops(0xD90, 0x0, 0x7FF7B7B02A84)	= 0 0
3471/0x68a0c: issetugid(0x0, 0x0, 0x0)	= 0 0
3472/0x68a0d: ioctl(0x2, 0x4004667A, 0x7FF7B7B02A14)	= -1 25
3472/0x68a0d: ioctl(0x2, 0x40487413, 0x7FF7B7B02A18)	= -1 25
3472/0x68a0d: mprotect(0x10840A000, 0x1000, 0x0)	= 0 0
3472/0x68a0d: mprotect(0x108411000, 0x1000, 0x0)	= 0 0
3472/0x68a0d: mprotect(0x108412000, 0x1000, 0x0)	= 0 0
3472/0x68a0d: mprotect(0x108419000, 0x1000, 0x0)	= 0 0
3472/0x68a0d: mprotect(0x108405000, 0xC8, 0x1)	= 0 0
3472/0x68a0d: mprotect(0x108405000, 0xC8, 0x3)	= 0 0
3472/0x68a0d: mprotect(0x108405000, 0xC8, 0x1)	= 0 0
3471/0x68a0c: getentropy(0x7FF7B9BD60E0, 0x20, 0x0)	= 0 0
3472/0x68a0d: mprotect(0x10841A000, 0x1000, 0x1)	= 0 0

```

3472/0x68a0d: mprotect(0x10841B000, 0xC8, 0x1)          = 0 0
3472/0x68a0d: mprotect(0x10841B000, 0xC8, 0x3)          = 0 0
3472/0x68a0d: mprotect(0x10841B000, 0xC8, 0x1)          = 0 0
3472/0x68a0d: mprotect(0x108405000, 0xC8, 0x3)          = 0 0
3472/0x68a0d: mprotect(0x108405000, 0xC8, 0x1)          = 0 0
3472/0x68a0d: mprotect(0x10841A000, 0x1000, 0x3)         = 0 0
3472/0x68a0d: mprotect(0x10841A000, 0x1000, 0x1)         = 0 0
3472/0x68a0d: issetugid(0x0, 0x0, 0x0)                  = 0 0
3471/0x68a0c: getattrlist("/Users/sedimav/Desktop/\320\236\320\241/LAB1/child\0", 0x7FF7B9BD6910,
0x7FF7B9BD692C)          = 0 0
3471/0x68a0c: access("/Users/sedimav/Desktop/\320\236\320\241/LAB1\0", 0x4, 0x0)          = 0 0
3471/0x68a0c: open("/Users/sedimav/Desktop/\320\236\320\241/LAB1\0", 0x0, 0x0)          = 3 0
3471/0x68a0c: fstat64(0x3, 0x7FE8DD704440, 0x0)          = 0 0
3472/0x68a0d: getentropy(0x7FF7B7B020E0, 0x20, 0x0)      = 0 0
3471/0x68a0c: csrctl(0x0, 0x7FF7B9BD6B1C, 0x4)              = -1 1
3471/0x68a0c: fgetattrlist(0x3, 0x7FF7B9BD6BD0, 0x7FF7B9BD6B30)      = 0 0
3471/0x68a0c: __mac_syscall(0x7FF81F0EFD45, 0x2, 0x7FF7B9BD6B30)      = 0 0
3471/0x68a0c: fcntl(0x3, 0x32, 0x7FF7B9BD6820)              = 0 0
3471/0x68a0c: close(0x3)                  = 0 0
3471/0x68a0c: open("/Users/sedimav/Desktop/\320\236\320\241/LAB1/Info.plist\0", 0x0, 0x0)          = -
1 2
3471/0x68a0c: proc_info(0x2, 0xD8F, 0xD)                  = 64 0
3472/0x68a0d: getattrlist("/Users/sedimav/Desktop/\320\236\320\241/LAB1/child\0", 0x7FF7B7B02910,
0x7FF7B7B0292C)          = 0 0
3471/0x68a0c: csops_audittoken(0xD8F, 0x10, 0x7FF7B9BD6B40)          = -1 22
3472/0x68a0d: access("/Users/sedimav/Desktop/\320\236\320\241/LAB1\0", 0x4, 0x0)          = 0 0
3472/0x68a0d: open("/Users/sedimav/Desktop/\320\236\320\241/LAB1\0", 0x0, 0x0)          = 3 0

```

```

3472/0x68a0d: fstat64(0x3, 0x7FBBDF04440, 0x0)          = 0 0

3472/0x68a0d: csrctl(0x0, 0x7FF7B7B02B1C, 0x4)          = -1 1

3472/0x68a0d: fgetattrlist(0x3, 0x7FF7B7B02BD0, 0x7FF7B7B02B30)      = 0 0

3472/0x68a0d: __mac_syscall(0x7FF81F0EFD45, 0x2, 0x7FF7B7B02B30)      = 0 0

3472/0x68a0d: fcntl(0x3, 0x32, 0x7FF7B7B02820)          = 0 0

3472/0x68a0d: close(0x3)                = 0 0

3472/0x68a0d: open("/Users/sedimav/Desktop/320\236\320\241/LAB1/Info.plist\0", 0x0, 0x0)      = -
1 2

3472/0x68a0d: proc_idtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel
access in action #11 at DIF offset 28

dtrace: error on enabled probe ID 1747 (ID 575: syscall::sysctl:return): invalid kernel access in action #11 at
DIF offset 28

dtrace: error on enabled probe ID 1719 (ID 177: syscall::read:return): invalid kernel access in action #13 at
DIF offset 68

dtrace: error on enabled probe ID 1719 (ID 177: syscall::read:return): invalid kernel access in action #13 at
DIF offset 68

dtrace: error on enabled probe ID 1717 (ID 179: syscall::write:return): invalid kernel access in action #13 at
DIF offset 68

dtrace: error on enabled probe ID 1719 (ID 177: syscall::read:return): invalid kernel access in action #13 at
DIF offset 68

nfo(0x2, 0xD90, 0xD)          = 64 0

3472/0x68a0d: csops_audittoken(0xD90, 0x10, 0x7FF7B7B02B40)          = -1 22

3471/0x68a0c: open("trace1.txt\0", 0x601, 0x1A4)          = 3 0

3471/0x68a0c: close(0x3)                = 0 0

3472/0x68a0d: open("trace2.txt\0", 0x601, 0x1A4)          = 3 0

3470/0x68a03: wait4(0xD8F, 0x0, 0x0)          = 3471 0

3472/0x68a0d: close(0x3)                = 0 0

3470/0x68a03: wait4(0xD90, 0x0, 0x0)          = 3472 0

```

Вывод

Реализовано распределение строк между двумя дочерними процессами с удалением гласных (латинских и русских) и записью результата в отдельные файлы с использованием исключительно системных вызовов.